

CASO: EsSalud Perú – Gestión de Calidad para la Historia Clínica Electrónica

Contexto:

El Ministerio de Salud del Perú ha desplegado un sistema nacional de **Historia Clínica Electrónica (HCE)**, conectando hospitales públicos en Lima, Cusco, Puno y Loreto. El sistema debe registrar diagnósticos, prescripciones, historial médico, citas y referencias entre establecimientos.

Problemas detectados antes de implementar herramientas de calidad:

Problema	Impacto operativo
Errores al mostrar medicamentos prescritos	Riesgo clínico, retraso en tratamientos
Citas duplicadas o solapadas	Congestión de servicios, reclamos de pacientes
Pruebas manuales en Excel y Word	Baja cobertura, pérdida de trazabilidad
Falta de conexión entre requisitos, pruebas y bugs	Difícil auditoría y priorización de errores
Falta de métricas	Desconocimiento del avance y efectividad de QA

 Solución adoptada:

Herramientas empleadas:

- **QMetry Test Management** (on cloud)
- **Selenium WebDriver** para automatización
- **Jira Software** como sistema de tickets y bugs

Funcionalidades utilizadas:

Herramienta	Características utilizadas en el caso
QMetry	- Gestión de ciclos de prueba - Reutilización de casos - Soporte a pruebas BDD - Integración nativa con Jira - Dashboards personalizables - Soporte para pruebas manuales y automatizadas
Selenium	- Automatización de pruebas regresivas - Validación en múltiples navegadores - Integración con QMetry y Jenkins
Jira	- Registro y seguimiento de bugs - Workflow de QA - Trazabilidad con historias de usuario y pruebas

Resultados logrados:

Indicador	Antes	Después
Cobertura de pruebas de regresión	~25% (solo pruebas críticas)	85% (automatizadas y manuales)

Tiempo de ejecución de pruebas	7 días por iteración	2 días con automatización y QMetry
Bugs detectados por los usuarios	Alta tasa de reportes (60+ por semana)	Reducción del 70% de bugs en producción
Trazabilidad de QA	No había visibilidad	Completa trazabilidad (Req ↔ Prueba ↔ Bug)
Auditorías y cumplimiento sanitario	Riesgo de sanción	Cumplimiento normativo con evidencia trazable

CASO: SmartBank – App Móvil de Banca Digital

Contexto:

SmartBank es un banco 100% digital que ofrece productos financieros vía app. Cada mes lanza nuevas funcionalidades (transferencias inmediatas, pagos QR, tarjetas virtuales). La experiencia del usuario y la seguridad son elementos críticos para su posicionamiento.

Problemas detectados antes de implementar soluciones de calidad:

Problema	Impacto operativo
Transferencias que no se completaban	Pérdida de confianza, quejas en redes sociales
Inconsistencia en login con huella	Riesgo de fraude y experiencia negativa
QA hacía pruebas manuales limitadas	No se detectaban regresiones
Cada área usaba herramientas diferentes	Aisladas: Excel, Google Sheets, correos
El equipo de negocio no validaba pruebas	Se producían entregas con funcionalidades incompletas

Solución adoptada:**Herramientas empleadas:**

- **Tricentis Tosca**
- **Micro Focus ALM Octane**
- **Azure DevOps (CI/CD)**

Funcionalidades utilizadas:

Herramienta	Características utilizadas en el caso
Tosca	- Modelado visual sin código (no-code) - Pruebas automatizadas de UI, APIs y mobile - Ejecución distribuida - Test Data Management
ALM Octane	- Gestión de historias, pruebas y bugs - Visibilidad de progreso en tableros Kanban - Soporte a DevOps pipelines - Métricas de cobertura, tasa de fallos, calidad por release
Azure DevOps	- Ejecución automática de pruebas Tosca en cada build - Integración con repositorio de código - Automatización del ciclo entrega-validación

Resultados logrados:

Indicador	Antes	Después
Bugs detectados en producción	30-40 por release	<10 bugs (mayoría no críticos)
Tiempo de regresión	5 días	1.5 días con Tosca y CI/CD
Participación de analistas de negocio	Muy baja	Alta: crean y validan pruebas con Tosca
Visibilidad de calidad	Reportes en Excel sin métricas claras	Dashboards en ALM con trazabilidad completa
Tiempo de entrega de versiones	6 semanas (incluyendo QA manual)	3 semanas (automatizado y ágil)

CASO: NASA – Verificación y Validación del Software de Navegación para Exploración Espacial

Organización: NASA Jet Propulsion Laboratory (JPL)

Proyecto: Sistema de Navegación Autónoma para Misión en Marte

Objetivo: Asegurar la calidad del software que controla la navegación del rover en Marte, sin intervención humana en tiempo real.

Contexto:

- La misión utiliza un **rover autónomo** que se desplaza en la superficie marciana.
- Las **comunicaciones tienen un retraso de 4 a 20 minutos**, por lo que el software debe tomar decisiones de navegación, evitar obstáculos, planificar rutas, y reaccionar a eventos ambientales **sin intervención humana**.
- Cualquier fallo podría poner en riesgo años de investigación y miles de millones de dólares.

Desafíos del proyecto:

Reto	Riesgo asociado
Software altamente sensible	Un error menor puede provocar el fallo total
Imposibilidad de corrección rápida	No hay acceso físico al sistema en Marte
Alta complejidad algorítmica	Machine Learning para interpretación del terreno
Validación de decisiones autónomas	Pruebas deben predecir decisiones del software
Ambientes simulados extremos	Necesidad de pruebas en condiciones marcianas

Solución adoptada:**Herramientas de calidad y prueba utilizadas:**

1. **Tricentis Tosca**
2. **Parasoft SOAtest y Virtualize**
3. **Micro Focus ALM Octane**
4. **Simuladores personalizados de terreno y clima marciano**
5. **GitLab CI/CD** para pipelines autónomos

Detalles de implementación:

Herramienta	Uso específico en el proyecto
Tricentis Tosca	Modelado de rutas, pruebas funcionales sin código, validación de flujos autónomos, automatización de regresión completa
Parasoft	Pruebas de APIs que reciben telemetría del rover, pruebas de comportamiento del software ante eventos extremos
ALM Octane	Gestión de requisitos NASA-JPL, trazabilidad entre código, pruebas, decisiones y logs espaciales
Simuladores	Reproducción de superficies marcianas, condiciones de luz, polvo, gravedad, etc. para pruebas integradas con Tosca
CI/CD GitLab	Automatización de pruebas por cada build y dataset de simulación

Comparativo Antes vs. Después

Métrica/Indicador	Antes (previo a automatización)	Después (con soluciones sofisticadas)
Cobertura de rutas simuladas	40% (limitada por tiempo humano)	98% cobertura de escenarios de navegación
Validación de decisiones autónomas	Manual y parcial	Totalmente automatizada, determinística
Tiempo de prueba de regresión	~3 semanas por cambio menor	<6 horas con simulación + Tosca
Trazabilidad de requisitos	Documentación aislada	100% trazabilidad en ALM Octane
Incidentes detectados post-prueba	No se podían identificar hasta después	Simulaciones detectan 99% de fallas posibles